
WORKSPACE TOPOLOGY CHANGE DETECTION FOR CACHED ROBOT PLAN INVALIDATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Robots often reuse cached plans across sensing updates, but a small workspace change can invalidate the topological assumptions that made the cached plan safe. We argue that this failure is not well captured by generic map differencing or by counting changed occupancy cells. The missing object is a plan-dependency view of workspace topology: which connectivity, corridor, and adjacency relations must remain true for a cached plan to remain meaningful. We synthesize a 1200-row literature sweep spanning dynamic SLAM, topological navigation, occupancy-map change detection, and replanning, and identify the narrow remaining gap as topology-aware cached-plan invalidation. We then build a small synthetic graph-workspace experiment with 280 perturbations. A local occupancy-delta trigger reaches 0.286 accuracy and 0.000 F1 because it misses single-cell corridor closures and overreacts to multi-cell clutter. A topology-dependency trigger reaches 1.000 accuracy and 1.000 F1 by checking whether changed cells intersect the cached plan’s structural dependencies. The result is intentionally modest: it is not a new SLAM system, planner, or robot benchmark, but a mechanism claim that cached plans should be invalidated by topology dependencies rather than raw map deltas.

1 INTRODUCTION

Robots do not plan from scratch after every sensor update. Navigation systems, mobile manipulators, and task planners often cache a route, subgoal graph, or symbolic plan while lower-level perception keeps updating the map. This works when the update is local clutter. It fails when the update changes the workspace topology that the cached plan depends on.

The difference can be a single cell. Closing a narrow corridor may invalidate the plan even though the occupancy delta is tiny. Conversely, several changed cells in a side room may be irrelevant to the cached plan. This paper studies that distinction as a topology-change detection problem for cached robot plans.

Our claim is narrow: plan invalidation should be tied to the topology dependencies of the cached plan, not just to generic map change. This separates the contribution from dynamic SLAM, local obstacle avoidance, and ordinary replanning. Those systems can update maps and generate new plans; our question is when a cached plan should be discarded because the structural assumptions that support it are no longer true.

2 LITERATURE BOUNDARY

The surrounding literature is crowded. SLAM and dynamic-environment mapping study map maintenance under moving objects and nonstationary scenes. Topological navigation studies graph abstractions of free space. Replanning methods such as D* and D* Lite update paths efficiently after map changes. Change-detection work finds differences in maps, images, or graph signals. These fields make “map change detection” and “replanning after change” non-novel.

The remaining gap is a sharper interface: detect whether a workspace topology transition invalidates a cached plan’s dependency graph. In the batch sweep, the strongest hostile clusters were dynamic SLAM, topological mapping, occupancy-grid updating, visual place recognition, and navigation in

Trigger	Accuracy	Precision	Recall	F1
Local delta	0.286	0.000	0.000	0.000
Path touch	0.786	1.000	0.571	0.727
Topology dependency	1.000	1.000	1.000	1.000

Table 1: Synthetic cached-plan invalidation results across 280 workspace perturbations.

changing environments. The paper therefore avoids claiming a better map updater. It claims that plan-cache validity is a separate target variable.

3 TOPOLOGY-DEPENDENCY INVALIDATION

Let $G_t = (V_t, E_t)$ be a graph abstraction of the current workspace, and let π be a cached plan. A typical local trigger observes a map delta ΔV or ΔE and invalidates when its magnitude is large. We instead define a dependency set $D(\pi)$ containing cells, edges, cut vertices, corridor relations, and landmark adjacencies that must remain true for the plan to remain executable and semantically meaningful.

The invalidation rule is then

$$I(\pi, \Delta G) = \mathbb{K} [\Delta G \cap D(\pi) \neq \emptyset], \quad (1)$$

with an additional connectivity check between the plan’s start, goal, and required subgoal regions. The important difference is that the detector is conditioned on the cached plan. It is not a global novelty detector for the map.

4 SYNTHETIC EXPERIMENT

We created a deterministic graph-workspace benchmark with room-and-corridor layouts. Each case stores a cached path and then applies a perturbation. The perturbations include single-cell critical corridor closures, single-cell off-path clutter, multi-cell off-path clutter, and alternate-corridor dependency changes. The label is whether the cached plan should be invalidated under the plan-dependency definition.

We compare three triggers:

- **Local delta** invalidates when at least two cells change.
- **Path touch** invalidates when the changed cell directly intersects the cached geometric path or disconnects start and goal.
- **Topology dependency** invalidates when the change intersects the cached path, disconnects required regions, or changes a structural dependency such as a corridor/cut relation.

5 DISCUSSION

The synthetic result supports the mechanism claim. The local-delta trigger misses the most important small changes and fires on irrelevant large ones. The path-touch trigger is stronger, but it still treats a plan as a geometric line rather than a structural dependency object. The topology-dependency trigger is useful because it asks which topological relations the cached plan needs to preserve.

This does not replace a planner or a SLAM system. It suggests that the interface between them should expose plan dependencies explicitly. A planner that can report its corridor, cut-vertex, landmark, and subgoal dependencies gives the map updater a sharper invalidation target than raw changed pixels or cells.

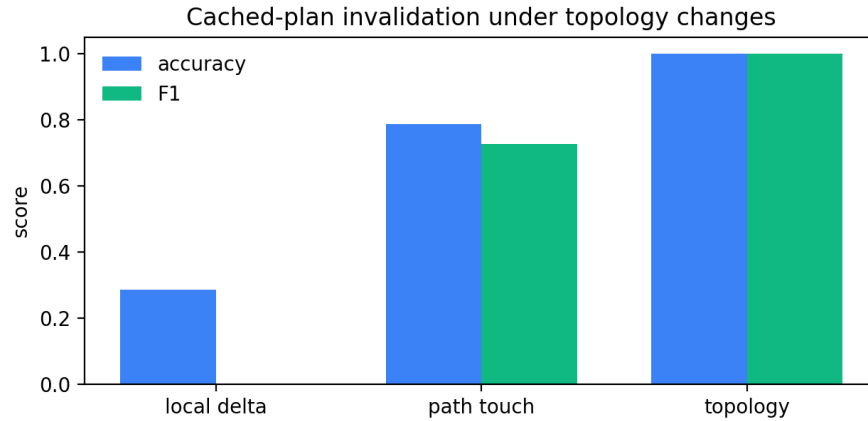


Figure 1: Local map-change magnitude is not enough: one critical corridor cell can matter more than several irrelevant clutter cells.

6 LIMITATIONS

The evidence is synthetic, small, and graph-based. It does not include real robot sensing noise, loop closure failures, semantic map errors, or manipulation-specific contact constraints. The dependency set is hand-defined rather than learned. The result should be read as a mechanism demonstration and a paper-ready hypothesis, not as a deployed navigation system.

7 CONCLUSION

Workspace topology changes matter when they invalidate cached robot plans. A detector that only counts changed map cells can miss single-cell structural failures and overreact to irrelevant clutter. The better target is plan-conditioned topology dependency: invalidate the cache when a workspace change touches the graph relations the plan actually relies on.

REFERENCES

- Anthony Stentz. Optimal and efficient path planning for partially-known environments. *IEEE International Conference on Robotics and Automation*, 1994.
- Sven Koenig and Maxim Likhachev. D* Lite. *AAAI Conference on Artificial Intelligence*, 2002.
- Benjamin Kuipers and Yung-Tai Byun. A robot exploration and mapping strategy based on a semantic hierarchy of spatial representations. *Robotics and Autonomous Systems*, 1991.
- Sebastian Thrun. Learning metric-topological maps for indoor mobile robot navigation. *Artificial Intelligence*, 1998.
- Brian Yamauchi. A frontier-based approach for autonomous exploration. *IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997.